

Betrachten Sie den in StudIP verfügbaren Programmcode `uap25-pro01-tram.zip`.

Aufgabe 1 : Implementierung der TRAM (7 Punkt(e))

Implementieren Sie die in der Vorlesung vorgestellte abstrakte Maschine. Die Klasse `Instruction` ist bereits vorgegeben und funktioniert wie folgt:

- Eine Instruktion besteht aus einem `opcode` und maximal drei Argumenten (`arg1`, `arg2`, `arg3`). Die Opcodes der einzelnen Instruktionen sind bereits durch statische Konstanten vorgegeben. Außerdem sind Konstruktoren vorhanden, um Instruktionen mit 0, 1, 2 bzw. 3 Argumenten zu erzeugen.
- Ein Programm wird als ein Array vom Typ `Instruction` dargestellt. Die Klasse `Instruction` enthält bereits die Beispiele aus der Vorlesung. Sie können diese nutzen, um die Funktion ihrer abstrakten Maschine zu testen.
- Sie können Ihre TRAM-Implementierung mit dem Assembler kombinieren. Dann können Sie auch Beispielprogramme im Verzeichnis `tramcode` zum Testen verwenden.
- Beachten Sie, dass die erweiterten Versionen von `store` und `load` verwendet werden, die jeweils **zwei** Argumente erwarten.

Schreiben Sie nun eine zweite Klasse `AbstractMachine`, welche die abstrakte Maschine aus der Vorlesung (also mit `Stack`, `TOP`, `PP`, `FP`, `PC`) implementiert und Programme (dargestellt durch ein Array vom Typ `Instruction`) ausführen kann. Beachten Sie zudem, dass zur Laufzeit mehrere Instanzen der abstrakten Maschine existieren können, die sich nicht untereinander beeinflussen dürfen. Verpacken Sie weiterhin die abstrakte Maschine als ausführbare JAR-Datei und ermöglichen Sie es, dass Dateien, die TRAM-Code enthalten, über die Konsole eingelesen und entsprechend ausgeführt werden können. Beispielaufruf auf der Konsole:

```
user@device:~$ java -jar tram.jar myprogram.tram
```

Aufgabe 2 : Debug-Modus (1 Punkt(e))

Erweitern Sie Ihre abstrakte Maschine um einen *Debug-Modus*. Verwenden Sie dazu das Logging Framework *Log4j*. Ist der Debug-Modus aktiv, so soll die Maschine nach jeder ausgeführten Instruktion ihren Zustand, d.h. die aktuellen Werte von `TOP`, `PP`, `FP` und `PC`

sowie den Inhalt des Stacks (nur bis TOP) ausgeben und/oder aufzeichnen. Eine Ausgabe könnte beispielsweise wie in Abbildung 1 dargestellt aussehen. Fügen Sie der ausführbaren

```
2018-04-19 12:19:04 DEBUG AbstractMachine:180 - After instruction = NOP; configuration = PC = 23; PP = 7; FP = 9; TOP = 14
Stack:
[0] = 28
[1] = 49
[2] = 0
[3] = 0
[4] = 0
[5] = 0
[6] = 28
[7] = 21 <-- PP (PP = 7)
[8] = 28
[9] = 0 <-- FP (FP = 9)
[10] = 2
[11] = 0
[12] = 0
[13] = 22 <-- FP_END (FP_END = 13)
[14] = 7 <-- TOP (TOP = 14)
```

Abbildung 1: Beispielhafte Ausgabe bei aktiviertem Debug-Modus der TRAM

JAR-Datei die Option hinzu den Debug-Modus an- bzw. abzuschalten. Beispielaufruf auf der Konsole:

```
user@device:~$ java -jar tram.jar -d myprogram.tram
```

Optional: Fügen Sie eine weitere Option hinzu, um die Debug-Ausgabe direkt in Dateien zu schreiben statt auf die Konsole.

Aufgabe 3 : Rekursiver Euklidischer Algorithmus (2 Punkt(e))

Schreiben Sie zum Testen ihrer abstrakten Maschine ein einfaches Programm, welches den euklidischen Algorithmus in seiner **rekursiven Variante** zur Berechnung des ggT zweier natürlicher Zahlen (inklusive 0) berechnet. Erzeugen Sie hierfür eine Datei `ggt.tram`, die die Maschineninstruktionen enthält und von der Konsole aus eingelesen werden kann. Beachten Sie, dass nach Ausführung des Programms TOP auf die Zelle des Stacks zeigt, in welcher sich das Ergebnis der Berechnung befindet.

Abgabeformat: Die Abgabe der Übungsblätter erfolgt über Stud.IP. Bitte beachten Sie die folgenden Hinweise. Abgaben, die nicht dem beschriebenen Format entsprechen, werden ignoriert und mit 0 Punkten bewertet:

- Bei **normalen Übungsblättern:**

Die Abgabe erfolgt als **einzelne PDF-Datei** (keine Ordner, kein ZIP-Archiv, nicht in mehreren Teilen, keine anderen Dateiformate). **Bitte vermerken Sie in dieser PDF-Datei gut lesbar Ihren Namen und Ihre Matrikelnummer.** Verwenden Sie folgendes Schema für den Dateinamen der PDF-Datei: `uap25-projxx-123456` (xx durch Nummer des entsprechenden Übungsblattes ersetzen, z.B. `ueb01` für das erste Blatt, und `123456` durch Ihre Matrikelnummer ersetzen).

- Bei **Übungsblättern mit Programmieraufgaben**:

Neben der evtl. vorhandenen PDF-Datei soll nur der Quellcode abgegeben werden (also keine `class`-Dateien o.Ä.). **Bitte vermerken Sie in jeder Quellcodedatei Ihren Namen und Ihre Matrikelnummer als Kommentar.** Packen Sie in diesem Fall alle benötigten Dateien (Quellcode und ggf. PDF-Datei) als **ZIP-Datei** mit dem Namen `uap25-projxx-123456.zip` (xx durch Nummer des entsprechenden Übungsblattes ersetzen, z.B. `ueb01` für das erste Blatt, und `123456` durch Ihre Matrikelnummer ersetzen).